NumPy & Pandas Essentials

Core Concepts & Patterns for Technical Interviews

NumPy

NumPy provides fast, vectorized operations on homogeneous arrays. It is the computational foundation that Pandas is built on top of.

Array Creation

```
import numpy as np

np.array([1, 2, 3])             # from a list
np.zeros((3, 4))               # 3x4 matrix of zeros
np.ones((2, 3))                # 2x3 matrix of ones
np.arange(0, 10, 2)            # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)           # 5 evenly spaced from 0 to 1
np.random.rand(3, 3)           # 3x3 random floats [0, 1)
np.random.randn(100)           # 100 samples, standard normal
np.eye(3)                      # 3x3 identity matrix
```

Shape & Reshape

```
arr = np.array([[1,2,3],[4,5,6]])
arr.shape        # (2, 3)
arr.reshape(3,2) # reshape — total elements must match
arr.flatten()    # collapse to 1D
arr.T            # transpose
```

Key: reshape() returns a view, not a copy. Modifying the reshaped array changes the original. Use .copy() for independence.

Indexing & Slicing

```
arr = np.array([10, 20, 30, 40, 50])
arr[1:4]         # [20, 30, 40]
arr[arr > 25]    # boolean indexing: [30, 40, 50]
arr[[0, 3]]      # fancy indexing: [10, 40]

# 2D
mat = np.array([[1,2],[3,4],[5,6]])
mat[0, 1]        # 2 (row 0, col 1)
mat[:, 0]        # [1, 3, 5] — all rows, first column
```

Aggregations

```
arr.sum()               # total
arr.mean()              # average
arr.std()               # standard deviation
arr.min(), arr.max()
arr.argmin(), arr.argmax() # INDEX of min/max
np.median(arr)
np.percentile(arr, 75)     # 75th percentile

# axis parameter
```

```
mat.sum(axis=0)          # sum down columns
mat.sum(axis=1)          # sum across rows
```

axis=0 means 'collapse rows' (operate down). axis=1 means 'collapse columns' (operate across). Think: the axis that disappears.

Vectorized Operations & Broadcasting

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b        # [5, 7, 9] — element-wise
a * b        # [4, 10, 18]
a ** 2       # [1, 4, 9]
np.dot(a, b) # 32 — dot product
np.sqrt(a)   # element-wise square root
```

Broadcasting: NumPy auto-stretches smaller arrays to match shapes. Example: mat + np.array([10, 20]) adds 10 to col 0, 20 to col 1 for every row.

np.where — Conditional Selection

```
arr = np.array([1, 2, 3, 4, 5])
np.where(arr > 3, arr, 0)  # [0, 0, 0, 4, 5]
np.where(arr > 3)          # returns indices: (array([3, 4]),)
```

Healthcare use: np.where(glucose > 126, 'abnormal', 'normal') — flag lab values above a clinical threshold.

Pandas

Pandas is a data manipulation library built on NumPy that gives you labeled, tabular data structures (Series and DataFrame) with built-in tools for cleaning, filtering, grouping, merging, and analyzing structured data.

Core Data Structures

```
import pandas as pd

# Series — 1D labeled array
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# DataFrame — 2D labeled table (used 95% of the time)
df = pd.DataFrame({
    'patient_id': [1, 2, 3],
    'age': [45, 67, 52],
    'glucose': [110, 180, 95]
})
```

Reading & Writing Data

```
df = pd.read_csv('patients.csv')
df = pd.read_excel('labs.xlsx')
df = pd.read_sql('SELECT * FROM encounters', conn)

df.to_csv('output.csv', index=False)
```

First Things You Do With Any DataFrame

```
df.head()          # first 5 rows
df.shape           # (rows, cols)
df.dtypes          # column types
df.info()          # non-null counts + dtypes
df.describe()      # summary stats for numeric columns
df.isnull().sum()  # missing values per column
```

Selection & Filtering

```
# Column selection
df['age']               # single column (Series)
df[['age', 'glucose']]  # multiple columns (DataFrame)

# Row selection
df.loc[0]               # by label (inclusive end)
df.iloc[0]              # by position (exclusive end)

# Filtering
df[df['age'] > 50]
df[(df['age'] > 50) & (df['glucose'] > 120)]  # use & | ~ not and/or
```

loc is label-based (inclusive end). iloc is integer-position-based (exclusive end). Interviewers test this distinction.

Handling Missing Data

```
df.isnull().sum()              # count missing per column
df.dropna()                    # drop rows with ANY null
df.dropna(subset=['glucose'])  # drop only if glucose is null
df.fillna(0)                   # fill all nulls with 0
df['glucose'].fillna(df['glucose'].median(), inplace=True)
```

Use median() over mean() for lab values — outliers (e.g., extreme glucose readings) skew the mean.

GroupBy — Most Tested Pandas Operation

```
# Average glucose by department
df.groupby('department')['glucose'].mean()

# Multiple aggregations
df.groupby('department').agg(
    avg_glucose=('glucose', 'mean'),
    patient_count=('patient_id', 'count'),
    max_age=('age', 'max')
)

# Most recent record per patient
df.sort_values('visit_date').groupby('patient_id').last()
# OR more explicit:
df.loc[df.groupby('patient_id')['visit_date'].idxmax()]
```

Sorting

```
df.sort_values('age')                                # ascending
df.sort_values('age', ascending=False)             # descending
df.sort_values(['dept', 'age'], ascending=[True, False])
```

Adding & Modifying Columns

```
df['bmi'] = df['weight'] / (df['height'] ** 2)
df['risk'] = df['glucose'].apply(lambda x: 'high' if x > 140 else 'normal')
df['age_group'] = pd.cut(df['age'], bins=[0,30,60,100],
                         labels=['young','mid','senior'])
```

Merging & Joining

```
# Inner join — only matching keys
pd.merge(patients, labs, on='patient_id', how='inner')

# Left join — keep all patients even without lab results
pd.merge(patients, labs, on='patient_id', how='left')

# Different column names
pd.merge(patients, labs, left_on='pid', right_on='patient_id')

# Concat — stacking DataFrames
pd.concat([df1, df2], axis=0)  # stack rows
pd.concat([df1, df2], axis=1)  # stack columns
```

Common pattern: 'Get all patients with their most recent lab result' — combines merge + groupby + sort.

Value Counts & Unique

```
df['department'].value_counts()  # frequency count, descending
df['department'].nunique()       # number of unique values
df['department'].unique()        # array of unique values
```

Apply & Map

```
# apply — run a function on each element/row/column
df['cat'] = df['glucose'].apply(lambda x: 'high' if x > 140 else 'normal')

# apply on rows (axis=1)
df['summary'] = df.apply(
    lambda row: f"{row['age']}yo, glucose={row['glucose']}", axis=1)

# map — Series only, good for recoding
df['sex'] = df['sex'].map({'M': 'Male', 'F': 'Female'})
```

Pivot Tables

```
df.pivot_table(
    values='glucose',
    index='department',
    columns='visit_type',
    aggfunc='mean'
)
```

String Operations

```
df['name'].str.lower()
df['name'].str.contains('diabetes', case=False)
df['icd_code'].str.startswith('E11')  # diabetes ICD-10 codes
df['notes'].str.split(',')
```

Date Operations

```
df['visit_date'] = pd.to_datetime(df['visit_date'])
df['year'] = df['visit_date'].dt.year
df['month'] = df['visit_date'].dt.month
df['days_since'] = (pd.Timestamp.now() - df['visit_date']).dt.days
```

Common Gotchas

Gotcha	Explanation
`df.unique()`	Does NOT exist. Use df['column'].unique() — it's a Series method.
`Chained assignment`	df[df['x'] > 5]['y'] = 10 may silently fail. Use: df.loc[df['x'] > 5, 'y'] = 10
`inplace + assignment`	df = df.dropna(inplace=True) sets df to None. Pick one: inplace=True OR reassignment.
`& vs and`	Use &, |, ~ for Pandas boolean ops, NOT Python's and/or/not.
`axis=0 vs axis=1`	0 = down rows (axis disappears). 1 = across columns.
`merge vs join`	merge is more flexible (specify keys). join defaults to index.